

Supplementary Material:

SAGE — Symbolic Action-Gating and Editing for LLM Task Planners

Abstract—This supplement provides the experimental detail behind the SAGE paper for reproducibility, organized around the paper’s multi-stream evidence rather than a single headline number. We give (i) the complete per-model plan-quality grid and per-difficulty breakdowns; (ii) the *leave-one-out* (leak-free) retrieval protocol, including the ceiling analysis that explains why goal-completeness ties on the standard benchmark; (iii) a harder, method-agnostic *multi-goal compound* benchmark on which SAGE’s completeness lead re-emerges; (iv) a failure-recovery *cost* study (LLM calls / latency / suffix length at equal recovery rate); (v) the *non-circular* runtime safety-monitor result, measured on a simulator-reported signal the verifier never sees; (vi) the verifier-portability (auto-induction) study across iTHOR, ProcTHOR and ALFWorld; and (vii) an account of the simulator setup and the validity of our execution metrics, which we report as negative-result contributions. We make no claim of “highest task-success.” All tables are auto-generated from the raw result CSVs; numbers therefore match the main paper exactly.

I. ROADMAP: CLAIMS TO EVIDENCE

This supplement is organized by experiment rather than by claim. Table I maps each main-paper claim (C1–C3), plus the cross-cutting generality evidence, to the comparison, the metric, and the sections that support it.

TABLE I
WHERE EACH CLAIM IS EVALUATED: COMPARISON, METRIC, AND SUPPORTING SECTIONS OF THIS SUPPLEMENT.

Claim	Comparison and metric	Evidence (this supplement)
C1: better plans where the benchmark discriminates	vs. 7 baselines; goal-completeness and plan validity/length	§IV (full grid), §V (compound tasks); context in §III
C2: higher step-level execution success	vs. Direct/Hierarchical, gate applied to all; step-success, precondition-violation count, human end-task	§VI (grounded exec.), §XI (safety monitor), §XII (metric validity), §XIV (human-judged)
C3: recovery	vs. whole-plan replan- cheapers; LLM calls, recovery rate, suffix length	§VII (recovery cost)
Generality (sup-ports all)	out-of-distribution, rule induction, edge, published baselines	§VIII (ProcTHOR), §IX (portability), §X (SOTA), §XV (on-device)

II. BENCHMARK DETAILS

The primary benchmark is a 75-task AI2-THOR (iTHOR) household set spanning four room types: 22 kitchen, 18 living-room, 15 bedroom, and 20 bathroom tasks. Tasks are stratified by horizon into 28 easy, 32 medium, and 15 hard instances. The set is built in two stages: an initial 38-task curated core (`eval_dataset_gt`), then 37 additional tasks produced by an object-grounded generation procedure: each candidate task is instantiated against the concrete object inventory of a target scene, then *simulator-verified* by executing a reference plan

in AI2-THOR and retaining only tasks whose goal conditions are reachable. This yields ground-truth reference plans used for all plan-quality metrics. For generalization we additionally evaluate on 70 tasks drawn from 40 ProcTHOR validation houses (unseen procedurally generated layouts, no re-tuning) and on the ALFWorld validation games, which use a distinct, out-of-distribution action vocabulary.

III. DATA LEAKAGE AND THE LEAVE-ONE-OUT PROTOCOL

The three memory-using methods (Few-Shot CoT, Hierarchical Few-Shot, and SAGE) retrieve few-shot exemplars from a *seed memory*. That seed memory is exactly the 38 `eval_dataset_gt` tasks, and those 38 tasks are a subset of the 75-task test set. A naive run therefore lets a memory method retrieve the *exact* reference plan for a test task it is currently being scored on. We treat this as a genuine confound rather than a footnote.

To remove it we added an opt-in *leave-one-out* (LOO) retrieval mode (`MEM_LEAVE_ONE_OUT=1`) that drops any exact-task match from the retrieval pool before answering, and re-ran the three memory methods $\times$ five models $\times$ 75 tasks under LOO. The non-memory baselines (Direct, CoT, Self-Refine, ReAct, Hierarchical) were already leak-free and are reused. All plan-quality numbers in the main paper and in this supplement use the LOO protocol; where useful we also report the 37 generated tasks as a strictly held-out, no-leak slice.

a) The base-set result is a tie, and we say so.: Under LOO, goal completeness on the base benchmark *saturates*: pooling over the three completed models, 52% (118/225) of task-instances are already scored 1.0 by every strong hierarchical baseline (Direct, Hierarchical, Hierarchical Few-Shot), leaving little headroom for a method gap. The aggregate is consequently a tie — in fact SAGE sits marginally behind Hierarchical Few-Shot on every base model (e.g. 0.870 vs. 0.878 on `qwen2.5:7b`), all within seed variance. This is a property of the benchmark’s completeness metric, not evidence that the method fails: a score maxed on roughly half the tasks *cannot* discriminate planners. Where headroom does remain (the hard tier), SAGE’s edge survives LOO (hard-task completeness 0.811 vs. 0.755 for the strongest hierarchical baseline, +0.056). We therefore (a) demote the verifier-computed `precondition_strict` to a diagnostic (it is circular, §XI), (b) report the tie on the base set, and (c) stress the regime where methods genuinely differ (§V).

IV. FULL PLAN-QUALITY RESULTS

Table II reports the complete offline grid under the LOO protocol: for each of the eight planning methods and five models we list mean goal completeness, strict precondition satisfaction (`precondition_strict`, reported as a diagnostic only), executability, mean total tokens, and mean LLM calls. The discriminating signal is not aggregate completeness (saturated, §III) but the difficulty profile: pooled over models (Table III), SAGE leads on the medium and hard tiers, where

one-shot and hierarchical plans systematically under-cover the sub-goals, while easy-task scores sit near ceiling for several methods.

TABLE II
FULL OFFLINE PLAN-QUALITY GRID (LEAVE-ONE-OUT PROTOCOL). PER MODEL: COMPLETENESS, `PRECONDITION_STRICT` (DIAGNOSTIC), EXECUTABILITY, MEAN TOTAL TOKENS, MEAN LLM CALLS. SAGE IS OUR PLANNER.

Method	Llama-3.2 3B					Qwen2.5 7B					Mistral-Nemo 12B					Qwen2.5 14B					Qwen2.5 32B				
	Compl.	P-strict	Exec.	Tokens	Calls	Compl.	P-strict	Exec.	Tokens	Calls	Compl.	P-strict	Exec.	Tokens	Calls	Compl.	P-strict	Exec.	Tokens	Calls	Compl.	P-strict	Exec.	Tokens	Calls
Direct	0.530	0.511	0.582	553	1.0	0.740	0.798	0.992	976	1.0	0.687	0.677	0.812	804	1.0	0.861	0.917	0.984	998	1.0	0.852	0.931	1.000	983	1.0
CoT	0.806	0.880	0.972	1355	1.0	0.832	0.903	0.996	1142	1.0	0.839	0.887	0.979	1156	1.0	0.886	0.920	0.979	1173	1.0	0.859	0.934	1.000	1180	1.0
Few-Shot CoT	0.571	0.593	0.987	451	1.0	0.622	0.571	0.953	424	1.0	0.653	0.608	0.967	414	1.0	0.603	0.496	0.880	303	1.0	—	—	—	—	—
Self-Refine	0.831	0.846	0.973	3753	3.7	0.824	0.902	0.992	2831	3.8	0.864	0.920	0.975	2788	3.6	0.839	0.917	0.969	1898	2.6	0.832	0.914	0.996	2659	3.5
ReAct	0.462	0.666	0.732	1132	2.9	0.733	0.669	0.946	2291	5.7	0.000	0.004	0.004	365	1.0	0.759	0.718	0.997	4302	9.6	0.753	0.685	0.999	3356	7.6
Hierarchical	0.760	0.873	0.925	3527	4.4	0.839	0.889	0.995	3001	3.9	0.880	0.889	0.966	3714	4.6	0.858	0.913	0.967	3072	4.0	0.876	0.883	1.000	3322	4.3
Hierarchical Few-Shot	0.821	0.877	0.933	7455	4.4	0.878	0.928	0.999	7075	4.9	0.858	0.927	0.984	7000	4.8	0.896	0.920	0.990	6951	4.9	—	—	—	—	—
SAGE (ours)	0.816	0.931	0.973	8553	5.3	0.870	0.966	0.996	6939	5.6	0.836	0.923	0.932	6902	5.5	0.883	0.961	0.998	6321	5.1	—	—	—	—	—

TABLE III
COMPLETENESS BY DIFFICULTY, POOLED OVER ALL MODELS (LEAVE-ONE-OUT). SAGE LEADS WHERE HEADROOM REMAINS (MEDIUM/HARD); EASY IS SATURATED.

Method	Easy	Medium	Hard
Direct	0.775	0.711	0.707
CoT	0.881	0.843	0.778
Few-Shot CoT	0.616	0.621	0.587
Self-Refine	0.870	0.839	0.776
ReAct	0.624	0.483	0.511
Hierarchical	0.892	0.840	0.755
Hierarchical Few-Shot	0.902	0.885	0.743
SAGE (ours)	0.835	0.884	0.811

V. HARDER BENCHMARK: MULTI-GOAL COMPOUND TASKS

Because base-set completeness is saturated (§III), we recover headroom with a harder benchmark built in a *method-agnostic* way. We compose same-scene single-goal tasks into 2- and 3-goal compound tasks (74 tasks; mean expected-object count 3.7 vs. 2.0 in the base set), with the reference plan formed by *concatenating* the already simulator-verified sub-plans. The construction references nothing about SAGE: it only lengthens the horizon and unions the goal set, so a planner must cover *all* sub-goals to score well.

Table IV reports the result. Once the ceiling is removed, SAGE’s completeness lead re-emerges and is large and consistent across all four models: +0.078 (qwen2.5:7b), +0.119 (qwen2.5:14b), +0.229 (llama3.2) and +0.057 (mistral-nemo) over the strongest baseline (Hierarchical Few-Shot). Crucially SAGE achieves this with *fewer* steps (12.0 vs. 17.8 on qwen2.5:7b) and higher precondition validity (0.943 vs. 0.893). The advantage is one of *brevity and validity*, not of per-step grounding: SAGE’s per-step hallucination rate is essentially the same as the baseline’s (0.53 vs. 0.55 on the compound tasks), so the baseline reaches comparable raw mentions only by emitting many more steps, while SAGE covers the same sub-goals with a shorter, precondition-valid plan. The effect is consistent across compound-2 (0.865 vs. 0.787) and compound-3 (0.837 vs. 0.759) splits, and the advantage is largest for the weakest model, where naive planners begin to bloat and drift. This confirms the ceiling hypothesis: the base-set tie was an artifact, and SAGE’s regime is hard, long-horizon, multi-goal tasks.

TABLE IV
GOAL COMPLETENESS ON THE HARDER MULTI-GOAL COMPOUND BENCHMARK (74 TASKS, LEAK-FREE). THE BASE-SET COMPLETENESS TIE DISAPPEARS: SAGE LEADS BY +0.057 TO +0.229 ACROSS THE FOUR MODELS, WITH FEWER STEPS AND HIGHER PRECONDITION VALIDITY (P-VALID).

Model	Direct	Hier-FS	SAGE	Δ	SAGE steps	SAGE p-val
qwen2.5:7b	0.718	0.776	0.854	+0.078	12.0	0.943
qwen2.5:14b	0.757	0.783	0.902	+0.119	12.0	0.937
llama3.2	0.629	0.499	0.728	+0.229	9.0	0.933
mistral-nemo	0.570	0.824	0.880	+0.057	12.7	0.952

VI. GROUNDED EXECUTION

Beyond offline plan quality, every plan is executed in the AI2-THOR simulator. Our primary execution metric is *step-level success* (exec_step_success): the fraction of a plan’s steps that the simulator accepts and executes without an environment-level error. Table V reports this per model for Direct, Hierarchical, and SAGE. Pooled over the five models, SAGE reaches 0.710 step success versus 0.602 (Direct) and 0.629 (Hierarchical). We are explicit about scope: absolute goal-level *task* success is *not* reliably measurable in our setup (see Section XII), because both automated task-success checkers proved unreliable. We therefore report step-level success together with the failure analysis as the execution-side evidence rather than a single headline task-success number.

TABLE V

GROUNDING STEP-LEVEL EXECUTION SUCCESS (EXEC_STEP_SUCCESS) IN AI2-THOR, PER MODEL. BOLD = BEST PER ROW.

Model	Direct	Hierarchical	SAGE
Llama-3.2 3B	0.565	0.613	–
Qwen2.5 7B	0.591	0.585	–
Mistral-Nemo 12B	0.547	0.622	–
Qwen2.5 14B	0.597	0.672	–
Qwen2.5 32B	0.708	0.652	–
Pooled	0.602	0.629	–

VII. FAILURE ANALYSIS AND RECOVERY COST

a) Failure taxonomy.: Table VI categorizes every failed execution (pooled over the five models, recovery enabled) into precondition violations, incomplete plans (goal not met), plan-construction errors, and simulator/infrastructure errors. The mechanistic signature of the symbolic verifier is clearest here: **precondition-violation failures fall to 0.27% of executions for SAGE versus 4.5% for Direct**, a $17\times$ reduction (and $\sim 7\times$ over Hierarchical). SAGE’s residual failures shift away from groundable precondition errors toward harder semantic “goal-not-met” cases, the failure class the verifier cannot repair, because it gates emitted plans but cannot synthesize a step the LLM never produced.

b) Recovery is measured as cost, not rate.: Because recovery rate saturates (every method recovers 100% of the fired injections at this difficulty), recovery rate cannot discriminate. We therefore measure the *cost* of recovery. We inject one mid-execution failure per task (at the scripted step where present, else the plan midpoint), force that step to fail to drive the planner’s recovery path, and record the LLM calls, latency, and suffix length spent to repair the plan and still reach the goal (held-out tasks, qwen2.5:7b, LOO, max_replan=3; rows counted only where the injection actually fired). SAGE recovers at 1.18 **LLM calls** on average, versus 2.87–3.71 for the whole-plan replanners (Hierarchical, Hierarchical Few-Shot, Self-Refine, ReAct): a $2.4\text{--}3.1\times$ reduction at equal (100%) reliability and lowest/near-lowest latency (1.13 s). The mechanism is sub-goal-local suffix repair: SAGE locates the failed sub-goal and regenerates only its suffix, preserving completed steps *and* untouched future sub-goals (verbose logs confirm e.g. a single repaired step preserving four untouched future steps), whereas baselines regenerate the whole remaining plan.

On the harder compound tasks the cost gap holds and widens: SAGE recovers at $2.6\text{--}3.3\times$ **fewer LLM calls** and up to $3.7\times$ lower latency (1.75 s vs. 6.49 s for Hierarchical Few-Shot), because whole-plan replan cost explodes with horizon (Hier-FS regenerates a ~ 12.6 -step suffix) while SAGE’s suffix repair stays bounded (~ 5.5 steps). Combined across both regimes: $2.4\text{--}3.3\times$ **fewer recovery LLM calls at equal recovery**. This is, to our knowledge, the first measurement of repair *cost* rather than repair success for embodied LLM planners.

TABLE VI

FAILURE TAXONOMY (SIM, RECOVERY ENABLED), COUNTS POOLED OVER FIVE MODELS, 375 EXECUTIONS PER METHOD.

Failure category	Direct	Hierarchical	SAGE
Precondition violation	17	7	1
Incomplete plan (goal not met)	20	11	5
Plan-construction error	0	5	1
Sim/infra error	0	1	0
Total failed (of 375)	37	24	7

TABLE VII

FAILURE-RECOVERY STUDY. AT EQUAL (100%) RECOVERY, SAGE REPAIRS AT $2.4\text{--}3.3\times$ FEWER LLM CALLS THAN WHOLE-PLAN REPLANNERS BY REGENERATING ONLY THE FAILED SUB-GOAL’S SUFFIX; THE GAP WIDENS ON THE HARDER COMPOUND TASKS.

Method	n	Recovery	Latency (s)	LLM calls	vs SAGE
SAGE (ours)	33	1.00	1.13	1.18	1.0 $\times$
Hierarchical Few-Shot	27	1.00	1.40	2.96	2.5 $\times$
Hierarchical	31	1.00	1.18	2.87	2.4 $\times$
Self-Refine	27	1.00	2.98	3.59	3.0 $\times$
ReAct	31	1.00	1.69	3.71	3.1 $\times$

VIII. GENERALIZATION TO PROCTHOR

On 70 tasks from unseen ProcTHOR validation houses, with *no* re-tuning of any component, SAGE retains the highest strict-precondition satisfaction (0.985) of all evaluated methods. As on the base set, completeness saturates near 1.0 for the hierarchical methods on these templated layouts, so this check speaks to plan *validity* transferring to unseen layouts rather than to a completeness win; precondition_strict is reported here as the verifier-side diagnostic, with the independent execution evidence in §XI. SAGE continues to gate out steps whose preconditions are not satisfied by the objects it has actually localized, even in procedurally generated rooms it never saw during development, indicating that the verifier’s precondition-checking signal, not memorized scene structure, drives the result.

IX. AUTO-VERIFIER TRANSFER (PORTABILITY STUDY)

The verifier SAGE *deploys* is hand-written (~ 250 lines of Python); we do *not* claim the deployed system learns its rules. We do, separately, show that those rules are not arbitrary hand-tuning but are *inducible* from interaction data, a portability study showing the safety monitor can be ported to a new action distribution from logged transitions rather than fresh authoring. Table VIII reports a probe-based induction pipeline (collect (pre-state, action, success) transitions, mine the conjunctive precondition per action, zero manual rules) run on three action ontologies. It recovers the hand-written reference in-domain (iTHOR, 14/16 literals matched), on unseen ProcTHOR layouts (13/16), and on ALFWorld (a domain with a completely different vocabulary: take, put, open, close) at 0.89 held-out predictive accuracy, recovering the core preconditions automatically (take: $\neg$ holding, open:openable $\wedge$ $\neg$ open, close:open, put:holding). The

TABLE VIII

CROSS-DOMAIN *portability* OF THE VERIFIER: THE SAME INDUCTION PIPELINE (ZERO MANUAL RULES) RECOVERS THE HAND-WRITTEN RULES IN-DOMAIN (iTHOR, 14/16), ON UNSEEN PROCTHOR HOUSES (13/16), AND ON ALFWORLD (0.89 HELD-OUT). THE DEPLOYED VERIFIER IS HAND-WRITTEN; THIS IS A SEPARATE INDUCIBILITY RESULT.

Action	iTHOR (in-domain)	ProcTHOR (unseen)	fail-recall	F1
	Induced precondition.	Induced precondition.		
Pick	$found \wedge \neg holding \wedge pickupable$	$found \wedge pickupable$	0.20	0.32
Open	$found \wedge openable$	$found \wedge openable$	0.39	0.53
Close	$found \wedge openable$	$found \wedge openable$	0.63	0.73
TurnOn	$found \wedge toggleable$	$found \wedge toggleable$	0.77	0.82
TurnOff	$found \wedge toggleable$	$found \wedge toggleable$	0.79	0.82
Place	$holding \wedge \neg toggleable \wedge receptacle$	$holding \wedge \neg toggleable \wedge receptacle$	0.78	0.88
PutIn	$holding \wedge receptacle$	$holding \wedge receptacle$	0.69	0.77
Match vs. ref	14/16	13/16		
Held-out pred-acc	0.859	0.683		
#transitions	702	723		

TABLE IX

GATE FAILURE-DETECTION ON 702 GROUNDED iTHOR TRANSITIONS AS THE RULE SET IS DEGRADED (RANDOM SUBSETS, 200 SAMPLES/LEVEL) AND WHEN REPLACED BY THE AUTO-INDUCED RULES. DEGRADATION IS SMOOTH; THE NON-EXPERT INDUCED SET MATCHES THE HAND-WRITTEN ONE.

Action	iTHOR (in-domain)	ProcTHOR (unseen)	fail-recall	F1
	Induced precondition.	Induced precondition.		
Pick	$found \wedge \neg holding \wedge pickupable$	$found \wedge pickupable$	0.20	0.32
Open	$found \wedge openable$	$found \wedge openable$	0.39	0.53
Close	$found \wedge openable$	$found \wedge openable$	0.63	0.73
TurnOn	$found \wedge toggleable$	$found \wedge toggleable$	0.77	0.82
TurnOff	$found \wedge toggleable$	$found \wedge toggleable$	0.79	0.82
Place	$holding \wedge \neg toggleable \wedge receptacle$	$holding \wedge \neg toggleable \wedge receptacle$	0.78	0.88
PutIn	$holding \wedge receptacle$	$holding \wedge receptacle$	0.69	0.77

X. SOTA BASELINES

Table X compares SAGE to two established neuro-symbolic / affordance-grounded planners: LLM+P (LLM translates the task to PDDL, solved by pyperplan) and SayCan (LLM scores affordance-filtered action candidates). On the same leave-one-out base grid as Table II (so these numbers match SAGE’s grid completeness, not a separate subset), SAGE attains substantially higher completeness (0.87–0.88) than both LLM+P (~ 0.71) and SayCan (~ 0.62). We report one caveat honestly: **SayCan’s precondition_strict = 1.0 is true by construction**, because our SayCan implementation uses the same symbolic verifier as its affordance filter, so it can only ever emit precondition-satisfying steps; its weakness is completeness (it frequently fails to reach the goal) rather than precondition validity. The comparison thus isolates SAGE’s gain as one of *goal completion* at matched precondition soundness.

same induction code, on three different ontologies, recovers sensible precondition logic in all three. Residual misses (e.g. Place’s ARRIVED literal) are probe-design-limited (the simulator auto-navigates on Place, masking the precondition), not learner-limited.

a) *Robustness to rule quality.*: A natural worry is that the headline gains depend on a carefully hand-tuned rule set. They do not. We stress-test the gate two ways and measure how well the degraded gate still separates real simulator failures from successes on the 702 grounded iTHOR transitions (failure-detection F1). (i) *Random rule deletion*: keeping a random fraction of the 13 atomic precondition rules degrades detection *smoothly and monotonically* — a random 75% retains 88% of full F1 (0.73 vs. 0.82), 50% gives 0.53 — with no single load-bearing rule and no brittle cliff (Table IX). (ii) *Replace hand rules with the auto-induced set*: the zero-manual-authoring induced rules *match or exceed* the hand-written set (F1 0.88 vs. 0.82, fail-recall 0.78 vs. 0.79); they edge ahead only by mining object-type affordances (e.g. *pickupable*, *receptacle*) that the minimal five-predicate hand set omits. The identical hand rules applied to unseen ProcTHOR layouts retain F1 0.77. Comparable rules are thus obtained *automatically*, *without an expert*, and partial rule sets degrade gracefully — so the result is not an artifact of expert rule-engineering.

TABLE X
SOTA COMPARISON: LLM+P AND SAYCAN VS SAGE. SAYCAN’S
PRECONDITION_STRICT= 1.0 IS BY CONSTRUCTION (VERIFIER AS
AFFORDANCE FILTER).

Method	Metric	Qwen2.5 7B	Qwen2.5 14B	Qwen2.5 32B
LLM+P	Compl.	0.770	0.713	0.638
	P-strict	0.963	0.965	0.976
SayCan	Compl.	0.620	0.607	0.620
	P-strict	1.000	1.000	1.000
SAGE	Compl.	0.870	0.883	–
	P-strict	0.966	0.961	–

XI. RUNTIME SAFETY MONITOR

This section provides the **independent, non-circular** validity evidence for SAGE. The metric here is `exec_step_success`, which is *simulator-reported*: it is the fraction of executed steps AI2-THOR itself accepts, and the verifier never sees it. Because this signal is independent of the verifier’s own `precondition_strict` score, an improvement under the gate cannot be a circular artifact of the verifier scoring its own output.

The verifier is LLM-free, so it runs *online* as a verify-before-execute monitor: before each step is dispatched to the robot, the verifier predicts precondition violations; flagged steps are repaired pre-emptively (`planner.replan`) and never executed; gate-OFF is the unchanged execute-then-replan path. Table XI contrasts gate-OFF and gate-ON on the held-out tasks (`qwen2.5:7b`, four planners). Enabling the gate raises simulator-reported step success for *every* planner, with per-planner deltas that scale monotonically with planner weakness: **Direct** +0.107 (20 invalid steps blocked), **CoT** +0.081, **Hierarchical** +0.041, and **SAGE** +0.016, the smallest lift for SAGE because it already verifies internally. This dose-response (biggest help to the weakest planner) is the signature of a real effect, not an artifact, and “prevented $\approx$ repairs” confirms nearly every blocked step is replaced by a repaired one. Pooled over Direct and Hierarchical, the gate also prevents 206 precondition-violating actions across four models before they reach the actuator (Table XI), turning the offline verifier into a deployable safety layer.

a) *Caveats.*: The current gate study is single-seed on `qwen2.5:7b` for the per-planner deltas; `exec_task_success` retains its known proxy-reliability caveat (§XII), so we lead with `exec_step_success`; and part of the gain is preemptive *repair* rather than pure blocking, so we frame the result as “monitor + local repair.”

TABLE XI
RUNTIME SAFETY MONITOR (VERIFY-BEFORE-EXECUTE).
`EXEC_STEP_SUCCESS` IS SIMULATOR-REPORTED AND INDEPENDENT OF
THE VERIFIER (NON-CIRCULAR). PER-PLANNER DELTAS: DIRECT +0.107,
CoT +0.081, Hier +0.041, SAGE +0.016. PREVENTED = UNSAFE
ACTIONS BLOCKED BEFORE ACTUATION.

Model	Prevented actions	Repairs fired	step-success	
			gate-off	gate-on
Llama-3.2 3B	54	26	0.589	0.605
Qwen2.5 7B	59	37	0.588	0.664
Mistral-Nemo 12B	52	30	0.585	0.628
Qwen2.5 14B	41	25	0.634	0.635
Total	206			

XII. SIMULATOR SETUP AND METRIC VALIDITY

Execution runs use AI2-THOR driven over a ZMQ socket by a custom `thor_server` process; the planner connects as a client, issues primitive actions, and reads back object/agent state. We define `step_success` as the fraction of emitted plan steps the simulator accepts (the action is applicable in the current world state and completes without an environment error).

We are explicit about why step-level success, not goal-level task success, is our reported execution metric, and we treat these as *negative-result contributions*: **both automated task-success checkers we implemented were unreliable**. (i) A reward-based proxy (`reward > 0`) gives a falsely high $\sim 94\%$ task-success ceiling, because AI2-THOR emits positive shaping reward for many partial interactions that do not actually satisfy the goal. (ii) A deterministic goal-condition checker is, conversely, far too strict: it scores only $\sim 8\%$ (3 of 38) of the *ground-truth reference plans* as successful, i.e. it fails even on the gold plans that by construction reach the goal. With one checker too lenient and the other too strict, neither produces a trustworthy absolute task-success number. We therefore lead with `exec_step_success` and the failure analysis as the execution-side evidence, and we flag absolute task success as an open measurement problem for this class of benchmark: a finding, not a bug.

XIII. EXTERNAL VALIDATION: EAI VIRTUALHOME

To check that SAGE’s gains are not an artifact of metrics produced by its own verifier, we evaluate on the independent *Embodied Agent Interface* (EAI) VirtualHome action-sequencing benchmark, scored by EAI’s *own* transition model (not ours). Without changing any SAGE code, we compare the verifier-on configuration against the SAGE-NoVerifier ablation over 305 tasks and three seeds (Table XII; cells are verifier on/off, %). Two effects are robust across seeds. The verifier removes *redundant* steps on every model (−2.7 to −7.4 points, largest on the strongest model — third-party evidence for the brevity-and-validity mechanism of §V), and it lifts *executability* for the small open models (+3.6/ +3.7 on 7B/3B) but *saturates* on 14B (−1.5). Correctness is essentially unchanged: the gate improves plan *validity*, not goal

TABLE XII

EXTERNAL VALIDATION ON EAI VIRTUALHOME (INDEPENDENT SCORER; MEAN OF THREE SEEDS). CELLS ARE VERIFIER ON/OFF (%). EXECUTABILITY RISES FOR THE SMALL MODELS AND SATURATES ON 14B; REDUNDANCY FALLS ON EVERY MODEL.

Model	Exec.	Correct	Afford. err	Redund.
qwen2.5:7b	58.6/55.0	23.0/19.8	30.1/31.0	10.2/12.9
llama3.2 (3B)	37.3/33.6	8.3/8.2	51.7/53.7	9.5/13.9
qwen2.5:14b	70.4/71.9	33.3/33.8	17.1/16.8	9.2/16.6

grounding. This regime-specificity (help concentrated on the small models) is exactly the on-device argument of §XV.

XIV. HUMAN-JUDGED END-TASK SUCCESS

Because both in-simulator goal-condition checkers are unreliable (§XII), an automated absolute task-success number would be untrustworthy. We therefore measure end-task success by human judgement on a *blinded, method-shuffled* subset. We execute {Direct, Hierarchical, SAGE} on **24 tasks** drawn from the simulator-verified ground-truth set (9 easy / 10 medium / 5 hard, spanning 9 kitchen, 8 living-room and 7 bedroom tasks) in AI2-THOR (qwen2.5:7b) and save, per run, the final RGB frame, the final object states, and the executed plan ($24 \times 3 = 72$ items). **Two annotators — one author and one non-author — independently** rate each item on {0, 0.5, 1} (achieved / partially achieved / not achieved) from the frame, final state, and plan, with method labels hidden and item order shuffled so neither the planner identity nor the other rater’s score is visible.

Table XIII reports the result. SAGE attains the highest human-judged end-task success, 0.53, versus 0.40 (Direct) and 0.28 (Hierarchical), pooling the two raters. Inter-rater agreement is *substantial*: 77.8% exact agreement, Cohen’s $\kappa = 0.65$, and linearly-weighted $\kappa = 0.75$. Finally, the zero-token partial-credit verifier score (which awards fractional credit for satisfied sub-goals) tracks the human consensus at Pearson $r = 0.82$ (mean absolute error 0.16), markedly better than the binary verifier verdict ($r = 0.75$); this supports the partial-credit checker as a cheap, non-circular proxy for the (expensive) human judgement.

TABLE XIII

HUMAN-JUDGED END-TASK SUCCESS ON A BLINDED, METHOD-SHUFFLED 72-ITEM SUBSET (QWEN2.5:7B), POOLED OVER TWO ANNOTATORS (ONE AUTHOR, ONE NON-AUTHOR). INTER-RATER COHEN’S $\kappa = 0.65$ (WEIGHTED 0.75). AUTO = THE ZERO-TOKEN PARTIAL-CREDIT VERIFIER SCORE (PEARSON $r = 0.82$ VS. HUMANS).

Method	Human end-task	Auto (partial)	n
Direct	0.40	0.41	24
Hierarchical	0.28	0.35	24
SAGE (ours)	0.53	0.58	24

XV. ON-DEVICE (EDGE) DEPLOYMENT

The zero-token verifier’s value is largest exactly where compute is scarcest: the small (3–7B) open-weight models an embodied agent can run *on-board* are also the ones that emit the most precondition-violating actions for the gate to block (§XI), and the verifier itself calls no model. We verify this on real edge silicon. We deploy the planner on an NVIDIA Jetson AGX Orin (64 GB unified memory, Jet-Pack 6 / L4T R36.4.3, CUDA 12.6, MAXN power mode with `jetson_clocks` locked), serving 4-bit (Q4_K_M) models with Ollama (0.30.10, ARM64+CUDA, GPU-resident: GR3D_FREQ 94–97% throughout). AI2-THOR is *not* installed on the device: only the planner and the verifier run on-board (planner-on-edge, sim-on-host), so this measures on-device *planning* feasibility and verifier overhead, a partial mitigation of the simulation-only limitation (L1).

We run a balanced 20-task subset (7 easy / 7 medium / 6 hard across 5 rooms, seed 0) of the 75-task benchmark, Direct vs. SAGE, on three model sizes, and run the identical subset/seed on the datacenter server for a parity check. Table XIV reports the result.

a) *The gate is free on the edge.*: The symbolic verifier’s per-plan overhead, isolated in an on-device micro-benchmark (200 repetitions per plan over all 75 reference plans, plan length median 4 / max 8), is **0.008 ms median**, **0.017 ms max** — six orders of magnitude below the multi-second on-device LLM latency. On an edge SoC where every LLM token is expensive, the deterministic quality/safety check costs effectively nothing.

b) *No quality loss, feasible cost.*: On-device plan quality matches the server within run-to-run seed variance ($|\Delta| \leq 0.02$ on the SAGE rows for all three models), so moving the decision-making to the edge incurs no plan-quality regression. The cost columns are the expected AGX-Orin profile: decode throughput $41 \rightarrow 13$ tok/s as the model grows 3B $\rightarrow$ 14B, peak RAM 7.2/14.4/27.2 GB (all comfortable on 64 GB), mean board power 40–48 W. SAGE’s extra LLM calls (5.5–7.3 vs. Direct’s 1) raise its end-to-end latency, the on-device cost of the completeness lift; the verifier component of that latency is the 0.008 ms above, i.e. negligible.

TABLE XIV

ON-DEVICE DEPLOYMENT ON A JETSON AGX ORIN (MAXN).
ON-DEVICE COST (DECODE THROUGHPUT, PLAN LATENCY, LLM CALLS)
AND EDGE-VS-SERVER QUALITY PARITY ON THE SAME 20-TASK
SUBSET/SEED. SAGE’S COMPLETENESS LIFT OVER DIRECT REPRODUCES
ON-BOARD (+0.12/ + 0.16/ + 0.13 FOR 3B/7B/14B) AND EDGE/SERVER
QUALITY AGREES WITHIN SEED VARIANCE. THE SYMBOLIC VERIFIER
ADDS 0.008 MS/PLAN (MEDIAN, SEE TEXT).

Model	Method	On-device cost (Orin)			Quality (edge / server)		
		tok/s	Lat. (s)	Calls	compl.	exec.	p-strict
qwen2.5:3b	Direct	41	2.3	1.0	0.78 / 0.71	0.96 / 0.94	0.97 / 0.94
	SAGE	33	8.3	7.3	0.90 / 0.88	0.99 / 0.99	0.96 / 0.96
qwen2.5:7b	Direct	23	3.9	1.0	0.76 / 0.78	0.98 / 0.99	0.88 / 0.88
	SAGE	17	9.6	6.2	0.92 / 0.92	0.99 / 0.99	0.98 / 0.98
qwen2.5:14b	Direct	13	7.0	1.0	0.81 / 0.79	0.97 / 0.97	0.96 / 0.95
	SAGE	10	20.1	5.5	0.94 / 0.94	0.99 / 0.99	0.97 / 0.98

XVI. PROMPT TEMPLATES

SAGE issues four LLM roles, each with the fixed system prompt below (verbatim from the released code). The placeholders {ACTIONS_STR}, {STEP_SCHEMA}, and {JSON_EXAMPLE} are filled at call time with the action vocabulary, the precondition schema, and a one-shot JSON output example. The accompanying *user* message carries the task text, the scene observation and visible objects, the top-*k* retrieved exemplars, and — for *refine/repair* only — the typed verifier violation block (§VII). Decoding is the model’s native default temperature.

(1) Decompose — high-level system prompt.

You are a household task planner (high-level).
Study the retrieved examples to understand sub-goal granularity, then
break the given task into 2-5 ordered sub-goals (short natural-language
phrases). Each sub-goal should be a distinct phase the robot will
execute end-to-end before moving on.

Return ONLY valid JSON -- no markdown, no explanation:
{ "subgoals": ["sub-goal 1", "sub-goal 2", ...] }

(2) Expand — low-level sub-goal expansion.

You are a household assistant robot (low-level executor).
You will receive ONE sub-goal and must expand it into concrete robot
action steps. Stay within this sub-goal -- do not anticipate later ones.

Available robot actions:
{ ACTIONS_STR }

{ STEP_SCHEMA }

Return ONLY valid JSON:
{ JSON_EXAMPLE }

(3) Refine — single in-block precondition repair (verifier-gated).

You are repairing a household plan. The previous attempt violated
preconditions defined by the action schema.

Available robot actions:
{ ACTIONS_STR }

{ STEP_SCHEMA }

Return ONLY valid JSON with the corrected steps for this sub-goal:
{ JSON_EXAMPLE }

(4) Repair — suffix-only edit of the failed sub-goal.

You are repairing a failed sub-goal in a household plan.
Generate ONLY the remaining steps to finish the CURRENT sub-goal.
Do NOT re-execute already-completed steps. Do NOT plan future sub-goals.

Available robot actions:
{ ACTIONS_STR }

{ STEP_SCHEMA }

Return ONLY valid JSON:
{ JSON_EXAMPLE }

Fig. 1. Verbatim system prompts for SAGE’s four LLM roles. The *refine* role (3) is the in-planning verifier-gated repair of Alg. 2 in the main paper; the *repair* role (4) is the suffix-only recovery edit (Alg. 3) that regenerates only the failed sub-goal’s remaining steps.

XVII. HYPERPARAMETERS, STATISTICS, AND REPRODUCIBILITY

a) Models.: Five open-weight models served via Ollama: llama3.2 (3B), qwen2.5:7b, mistral-nemo (12B), qwen2.5:14b, and qwen2.5:32b-instruct. We run 3 random seeds for the three smaller models (3B–12B) and a single seed for the two largest (14B, 32B) due to compute cost. Each model is queried at its native default temperature; a warm-up call precedes every run to exclude cold-start latency from cost measurements.

b) Planner configuration.: SAGE uses `max_refines=1` (a single local repair per failed sub-goal); increasing it converges toward Self-Refine behavior and erodes the token-cost advantage, so it is fixed at 1 for

all reported runs. The symbolic verifier makes *zero* LLM calls and runs in $O(|\pi|)$ time. Memory retrieval uses the leave-one-out protocol (§III) for all reported plan-quality numbers.

c) Statistics.: All confidence intervals and significance tests use the Python standard library only (no SciPy). For paired method comparisons we use a sign-flip permutation test (2×10^4 permutations, two-sided, paired per task with seeds averaged before pairing). Task-level bootstrap 95% confidence intervals use 10^4 resamples. RNGs are seeded for reproducibility.

d) Software / data dependencies.: The Ollama backend uses the non-streaming `/api/chat` endpoint. ALFWorld transition logging requires TextWorld 1.7.0 under Python 3.13, which needs a small patch to the `textgen eval()` deriva-

tion (the stock code is incompatible with 3.13); LLM+P uses `pyperplan` as the symbolic solver. Exact run commands for each table are recorded alongside the result directories (`results/`).

e) Released artifacts.: We release the 75-task benchmark and its simulator-verified reference plans, the compound-task generator (§V), the leave-one-out retrieval mode (`MEM_LEAVE_ONE_OUT`), the `--inject-fail` failure-recovery harness (§VII), the `--verify-gate` runtime-monitor harness (§XI), and the verifier-portability induction pipeline (iTHOR/ProcTHOR/ALFWorld).

Limitations

- **Simulation only.** All *embodied execution* is in AI2-THOR / ProcTHOR / ALFWorld; real-robot deployment is future work, targeting a UFACTORY xArm for tabletop manipulation. We do run the *planner* on edge hardware (Jetson AGX Orin, §XV), a partial mitigation: the decision-making runs on representative on-board silicon even though actuation remains simulated.
- **ALFWorld is verifier-only.** The verifier-portability result on ALFWorld stands (rules induced at 0.89 held-out), but running SAGE’s *planner* end-to-end on ALFWorld is future work: ALFWorld’s action vocabulary (`clean/heat/cool`) is not in our action set, a domain action-vocabulary gap that planning, not the verifier, must bridge.
- **Saturated completeness.** On the standard benchmark goal completeness saturates (52% of instances at 1.0), so the base-set result is an honest tie; the method’s advantage is shown on the harder compound benchmark (§V).
- **Recovery rate saturates.** Every method recovers 100% of injected failures at this difficulty, so recovery *rate* does not discriminate; we therefore report recovery *cost* (LLM calls / latency / suffix length, §VII).
- **Single-seed largest models.** The 14B and 32B results are single-seed; their CIs are therefore not estimated across seeds, and the per-planner safety-gate deltas are currently single-seed on `qwen2.5:7b`.
- **Diagnostic-only precondition_strict.** It is computed by the same verifier SAGE filters with (circular), so we report it only as a diagnostic; as a predictor it is weak (AUC ≈ 0.63 for step-level executability, ≈ 0.56 for goal success). The independent validity evidence is the non-circular safety-monitor result (§XI).